COMP2040 – Operating Systems

Lab 02: Processes, Inter-Process Communication and Threads

Overview

This lab turns the Week 3 process and thread material into behaviour you can observe: you will create a child process, replace the program it runs, pass data between processes through a pipe and through shared memory, and split a small calculation between two threads.

Complete the `TODO` sections in the starter files and keep the given file names, otherwise the commands below will not work as written. If you are new to C, read the *C Primer for Lab 02* and run `c_concepts_arrays_pointers.c` first. Output from two processes may appear in either order, and your PIDs will differ from everyone else's: report what you observed.

Submission

Upload two files to Canvas before the deadline shown there:

1. `StudentID_Lab02.pdf` – a short report;
2. `StudentID_Lab02.zip` – the six completed `.c` files (no executables).

For each task, give the commands you used, the relevant terminal output (as text where possible) and short answers to the questions; two to four sentences is usually enough. There is no need to paste your source code.

1 Create and collect a child process

`task1_fork.c` declares `int value = 10`. Complete it so that the child sets its own `value` to `99` and prints its PID, its parent's PID and `value`, while the parent waits for the child with `waitpid()` and then prints its own `value`. The child should report `99`; the parent should still report `10`.

```
gcc -std=c11 -Wall -Wextra task1_fork.c -o task1_fork
./task1_fork
```

Include in your report.

(a) The compile command and both output lines.
(b) What does `fork()` return in the parent, in the child, and on failure?
(c) Why did changing `value` in the child not change the parent's `value`?
(d) What does `waitpid()` prevent the parent from doing too early?

2 Replace the child's program

`task2_worker.c` is complete: it prints its PID and the word given on its command line. Complete `task2_fork_exec.c` so that the child prints its PID and then runs `task2_worker` with `execl()`, while the parent collects the child with `waitpid()` and prints its exit status. The PID printed before `execl()` should match the worker's PID. Build both programs in the same directory:

```
gcc -std=c11 -Wall -Wextra task2_worker.c -o task2_worker
gcc -std=c11 -Wall -Wextra task2_fork_exec.c -o task2_fork_exec
./task2_fork_exec
```

Include in your report.

(a) The compile commands and the program output.
(b) Did `execl()` create another process? Use the PIDs as evidence.

(c) Why does the child call `_exit()` only if `execl()` fails?
(d) What is the difference between `fork()` and `exec()` in this example?

3 Send a message through a pipe

In `task3_pipe.c`, `fd[0]` is the read end of the pipe and `fd[1]` the write end. Complete it so that the child writes the message with `write()`, the parent reads it with `read()` and prints it, each process closes the end it does not use, and the parent collects the child with `waitpid()`. The parent should print `Parent received:  Hello from the child`.

```
gcc -std=c11 -Wall -Wextra task3_pipe.c -o task3_pipe
./task3_pipe
```

Include in your report.

(a) The compile command and the received message.
(b) Which pipe end does each process use, and which end must each one close?
(c) Why can the parent read data the child wrote after `fork()`, when the integer in Task 1 was private?
(d) Is this pipe one-way or two-way? What would you add for two-way communication?

4 Compare private and shared memory

Before calling `fork()`, `task4_shared_memory.c` creates an ordinary local integer and a shared integer (with `mmap()`), both set to `10`. Complete it so that the child sets *both* values to `99` and the parent waits for the child before the values are printed. The parent should print `private=10 shared=99`.

```
gcc -std=c11 -Wall -Wextra task4_shared_memory.c -o task4_shared_memory
./task4_shared_memory
```

Include in your report.

(a) The compile command and the program output.
(b) Why did only the shared value change in the parent?
(c) How does this form of inter-process communication differ from the pipe in Task 3?
(d) If both processes wrote the shared integer at the same time, what could go wrong?

5 Compute a matrix with two threads

`task5_threads.c` multiplies $A = \left(\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right)$ by $B = \left(\begin{smallmatrix} 5 & 6 \\ 7 & 8 \end{smallmatrix}\right)$. Complete `compute_row()` and the `pthread_create()` / `pthread_join()` sections so that **two threads** each compute one row of `result` and write only to that row. Do not add a mutex: the rows never overlap (mutexes are Week 5 material). The expected rows are `19 22` and `43 50`.

```
gcc -std=c11 -Wall -Wextra -pthread task5_threads.c -o task5_threads
./task5_threads
```

Include in your report.

(a) The compile command and the resulting matrix.
(b) Which data do the threads share? What work is assigned to each thread?
(c) Why must the main thread call `pthread_join()` before printing the result?
(d) Give one benefit and one cost of using separate processes instead of threads here.